The Autonomous SDLC Factory

Master Blueprint & Architecture Agreement

1. The Core Architecture: "The Hybrid Adapter"

The foundational philosophy of this system is the **Hybrid Adapter Architecture**. We are splitting the brain into two distinct halves to avoid the trap of rebuilding complex UIs from scratch, while strictly protecting against third-party lock-in.

- **The Source of Truth (Django):** The "Company Ledger." It owns the database, webhook routing, domain business logic, long-term memory, and acts as the true CEO connecting the outside world to the factory floor.
- **The UI & Orchestrator (Paperclip):** Acts *only* as the company dashboard. It manages the org chart, routes tasks, tracks budgets, and holds the approval gates. Django interacts with it purely via API.

2. The Final Tech Stack (What We Keep & Why)

Layer	Tool	The Architectural Justification
Business Logic & Ledger	Python + Django REST (DRF)	The ultimate foundation. Handles custom webhooks, complex state relations, and database migrations effortlessly. This is the unshakeable core.
Orchestrator Adapter	Paperclip	Provides the "Company Metaphor" (budgets, org charts, dashboards) for free. Used purely as an adapter to save months of UI/UX development.
Task Queue	Celery + Redis	Perfect for async jobs. When Django needs to trigger a long-running agent task, Celery handles it without blocking the main application thread.
Memory Vault	PostgreSQL + pgvector	A unified data layer. Stores structured records (Tickets, ExecutionLogs) while `pgvector` natively handles vector embeddings for architectural memory. Zero extra DB services needed.
	Docker	

Layer	Tool	The Architectural Justification
Execution Sandbox		Mandatory safety net. Agents execute code in strictly isolated containers to prevent catastrophic file system overwrites or rogue commands.
The Workforce	Claude Code + Codex	Claude handles the deep implementation and complex specs. Codex/OpenAI acts as an independent QA reviewer to catch blind spots.
CI/CD & Delivery	GitHub Actions	Industry standard. Handles automated tests, staging deployments, PR merges, and secrets management flawlessly.
Monitoring	Sentry	The watchtower. Catches production errors and fires webhooks straight into Django to automatically spawn bug tickets.
The Human Gate	Telegram Bot	High-mobility governance. Pings your phone for manual "Approve/Reject" decisions at critical pipeline junctions.

3. The Graveyard (What We Rejected & Why)

A strong system is defined by what it excludes. Here is exactly why we rejected these popular tools:

- **MetaGPT:** It is a "Project Factory," not a "Living Company." It lacks the continuous operational loop (Deploy → Monitor → Bug → Auto-Ticket) and granular budget governance required for a 24/7 autonomous company. It generates artifacts beautifully but fails at continuous enterprise operations.
- **Apache Kafka:** Severe over-engineering. Kafka is designed for millions of events per second. For an organization of ~10 agents processing dozens of tickets, Redis Pub/Sub provides all the event-driven power needed without the crushing DevOps overhead.
- **LangGraph & n8n:** Orchestrator overlap. Django is already routing webhooks, and Paperclip handles cyclic state natively via ticket reassignment. Adding node-based workflow engines creates duplicated state, duplicated routing, and debugging nightmares.
- **Pure Django Custom Build:** The "Platform Trap." Rebuilding Paperclip's dashboard, org chart, budget caps, and human-in-the-loop UI from scratch in Django would take months before a single AI feature is shipped.

4. The Infinite Loop Workflow

This is the fully closed-loop software delivery cycle, featuring autonomous feedback and iteration.

1. **Idea/Ticket Intake:** Sentry catches a bug OR a user submits a feature request. Django creates the Ticket.
2. **Market Validation/Research:** Claude Agent gathers context and writes a brief.
3. **Architecture & Spec:** Architect Agent designs DB models and API specs.
4. **Human Approval Gate:** Telegram pings you to approve the architecture.
5. **Task Decomposition:** Paperclip breaks the spec into actionable sub-tickets.
6. **Implementation:** Claude Code agents write the code inside Docker sandboxes (on isolated Git branches).
7. **Unit Testing & QA:** Codex/Review Agent runs `pytest` and reviews the PR. *(Cyclic loop: fails → bounces back to implementation).*
8. **Security Audit:** Specialized agent scans for vulnerabilities.
9. **Human Approval Gate:** Telegram pings you to review the final PR.
10. **Integration & Deploy:** GitHub Actions merges to main and deploys to staging/production.
11. **Observe & Monitor:** Sentry monitors the live application.
12. **Loop:** Sentry detects an anomaly, fires a webhook to Django, and the cycle repeats.

5. The 72-Hour Spike (Immediate Execution Plan)

Before building the 13-step loop, we must empirically validate the Django ↔ Paperclip bridge to ensure Paperclip is stable enough for production.

The Goal: Build a microscopic, end-to-end slice to validate orchestration, logging, and approval gates.

The Spike Pipeline:

1. **Django Creates Ticket:** A dummy ticket is generated in the PostgreSQL database.
2. **API Handoff:** Django pushes the ticket to Paperclip via API.
3. **Execution:** Paperclip assigns a basic Research Agent to execute a simple script inside a Docker container.
4. **The Ledger:** The agent returns the artifact. Django records the `prompt_cost`, `duration`, and result in the PostgreSQL `ExecutionLog` table.
5. **The Gate:** The Telegram bot pings your phone: *"Approve artifact?"*

Daily Breakdown:

- **Day 1 (The Data Foundation):** Spin up the Django project. Write the core models: `Ticket`, `AgentRun`, `ExecutionLog`, and `ApprovalGate`.
- **Day 2 (The API Adapter):** Set up local Paperclip. Write the Django service layer to sync the `Ticket` model to a Paperclip task.
- **Day 3 (The Execution & Gate):** Configure a basic Docker container for Claude Code. Run a test ticket, log the cost to the DB, and wire up the Telegram approval ping.

"By wisdom a house is built, and through understanding it is established."